SCHOLARSHIP CANDIDATE SELECTION

AND ALGORITHM VISUALIZER

PROJECT REPORT

Submitted By:

Team Members:

N.Kammakshi Lovkkyaa-25MVCSMR0691

N.Srividya-25MVCSMR0693

G.Sarthika-25MVCSMR0686

D.Pranitha-25MVCSMR0639

College:

MallaReddy Vishwavidyapeeth

Department:
CSE-AIML

Academic Year: 2026–27

1. Abstract & Introduction

Abstract

The Scholarship Candidate Selection and Algorithm Visualizer is an interactive web application designed to demonstrate how Design and Analysis of Algorithms (DAA) techniques can be applied to scholarship candidate selection. The system accepts student information through a structured file upload, validates the data, checks eligibility constraints, ranks eligible candidates, and performs scholarship selection using Sorting, Greedy, and Dynamic Programming techniques.

A key feature of the system is the Live Algorithm Activity panel, which allows users to observe what the algorithms are doing while the main dashboard remains visible. The application also compares algorithm outcomes using measures such as total benefit, budget utilization, number of selected candidates, and execution time.

Introduction

Scholarship selection can involve multiple constraints, including academic performance, financial need, category-specific eligibility rules, parent employment conditions, scholarship budget, and the number of available scholarships. Processing these factors manually can be time-consuming and may make the decision process difficult to explain.

This project models the selection process as an algorithmic optimization problem. It separates eligibility checking from ranking and optimization, making the process transparent and suitable for demonstrating fundamental DAA concepts.

2. Problem Statement, Objectives & Scope

Problem Statement

Develop an interactive system that can process scholarship applicant data, apply defined eligibility constraints, rank eligible candidates, and select candidates within scholarship budget and capacity constraints while clearly demonstrating the operation of Sorting, Greedy, and Dynamic Programming algorithms.

Objectives

- Provide a simple file-based method for uploading student data.
- Validate uploaded records and identify missing, duplicate, or invalid data.
- Apply configurable scholarship eligibility rules.
- Rank eligible students using a transparent scoring method.
- Implement and visualize a real sorting algorithm.
- Implement a Greedy selection strategy under scholarship constraints.
- Implement Dynamic Programming for constrained optimization.
- Explain why candidates are selected or rejected.
- Compare algorithm results and performance.
- Provide an interactive dashboard suitable for demonstration and experimentation.

Scope

The project focuses on an academic prototype for scholarship candidate selection. It supports fictional or appropriately authorized student datasets, configurable eligibility rules, budget and scholarship-slot constraints, algorithm visualization, and result comparison. The system is intended as a decision-support and educational tool rather than an automated replacement for official scholarship authorities.

3. Features & Functional Requirements

Major Features

- CSV-based student data upload with downloadable sample template.
- Optional XLSX support.
- Data validation and preview before processing.
- Eligibility checking using configurable rules.
- Student ranking using marks and financial-need criteria.
- Merge Sort visualization.
- Greedy scholarship selection.
- Dynamic Programming optimization.
- Live right-side algorithm activity panel.
- Final selection with reasons.
- Greedy versus DP comparison.
- Fairness and coverage statistics.
- Interactive algorithm playground.
- Demo mode with fictional student data.

Functional Requirements

- The system shall accept student data through a structured CSV file.
- The system shall validate required columns and values.
- The system shall display a preview of uploaded records.
- The system shall allow administrators to configure scholarship constraints.
- The system shall identify eligible and ineligible candidates.
- The system shall calculate a transparent candidate priority score.
- The system shall execute the implemented algorithms on the uploaded dataset.
- The system shall display algorithm activity without replacing the main dashboard.
- The system shall display final selected candidates and decision explanations.
- The system shall provide comparative performance information.

4. Algorithms & Methodology

Overall Methodology

The processing pipeline follows: Student File Upload → Data Validation → Eligibility Check → Ranking/Sorting → Greedy Selection → Dynamic Programming → Final Selection → Algorithm Comparison.

4.1 Sorting

Merge Sort is used to rank eligible candidates according to the selected criterion, such as marks, financial need, or a combined score. Merge Sort follows a divide-and-conquer strategy and has O(n log n) time complexity in the best, average, and worst cases.

The interface visualizes comparisons and movements of candidates so that the sorting process can be observed rather than only displaying the final sorted list.

4.2 Greedy Algorithm

The Greedy approach repeatedly considers the highest-priority feasible candidate and selects that candidate when eligibility, scholarship capacity, and remaining budget constraints are satisfied. This provides a straightforward and fast selection strategy. Its result depends on the defined greedy criterion and does not necessarily equal the globally optimal combination for every constrained optimization problem.

4.3 Dynamic Programming

Dynamic Programming is used when the selection problem can be represented using overlapping subproblems and optimal substructure. Candidate benefit, scholarship cost, total budget, and scholarship capacity can be incorporated into the optimization state. The system reconstructs the selected candidate combination from the computed optimal result.

The raw DP matrix is not shown on the main dashboard. Instead, the application presents understandable decision steps through the Live Algorithm Activity panel.

4.4 Candidate Scoring

The system uses a transparent weighted score based primarily on academic performance and financial need. The academic and financial-need weights can be configured. Caste/category and parent government-job status are treated primarily as eligibility or constraint information according to the configured scholarship rules, rather than being assigned arbitrary hidden ranking points.

5. Technologies & Implementation

Technology Stack

- Frontend: React with Vite
- Styling: Tailwind CSS
- Charts and visualization: Recharts
- Backend: Python with Flask (or Node.js where appropriate)
- Data storage: SQLite or local JSON
- Input format: CSV, with optional XLSX support

Implementation Modules

- File Upload and Validation Module
- Student Data Preview Module
- Eligibility and Constraint Module

- Scoring and Sorting Module
- Greedy Selection Module
- Dynamic Programming Module
- Live Algorithm Activity Module
- Results and Explanation Module
- Algorithm Comparison Module
- Dashboard and Visualization Module

Input Data Format

The primary student file contains the following columns:

- Student_ID
- Student_Name
- Marks
- Family_Income
- Caste_Category
- Parent_Government_Job
- Scholarship_Amount

6. Results, Testing & Performance

Expected Results

The completed system should successfully process a student dataset from upload through final scholarship selection. It should identify invalid records, apply eligibility rules, rank eligible students, execute the Greedy and Dynamic Programming approaches, and present the resulting selections with explanations.

Testing

- Valid CSV upload test.
- Missing-column test.
- Invalid marks and income test.
- Duplicate Student ID test.
- Eligibility boundary test.
- Budget constraint test.
- Scholarship-slot constraint test.
- Sorting correctness test.
- Greedy selection test.
- Dynamic Programming optimization test.
- Comparison and visualization test.

Performance Metrics

Performance can be evaluated using execution time, number of comparisons or decisions, total benefit achieved, budget utilization, number of candidates selected, and scalability with increasing dataset size. Exact measurements should be inserted after implementation and testing.

7. Conclusion, Future Scope & References

Conclusion

The Scholarship Candidate Selection and Algorithm Visualizer provides an interactive way to demonstrate the practical application of DAA techniques to a constrained selection problem. By combining eligibility rules, transparent scoring, Sorting, Greedy selection, and Dynamic Programming, the system makes the algorithmic decision process visible and understandable.

The Live Algorithm Activity panel is a central usability feature because it allows users and judges to observe algorithm decisions without losing access to the main dashboard and current results.

Future Scope

- Integration with authenticated institutional scholarship databases.
- Support for additional scholarship schemes and rule templates.
- More advanced optimization objectives and multi-constraint models.
- Expanded accessibility and multilingual support.
- Detailed audit logs for authorized administrators.
- Additional algorithm implementations for comparative study.
- Cloud deployment and role-based access control.

References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. – Introduction to Algorithms.
- Kleinberg, J., & Tardos, É. – Algorithm Design.
- Official documentation for React, Vite, Tailwind CSS, Flask, SQLite, and Recharts.
- Institutional scholarship guidelines or official scheme documentation, where applicable.